

redis HashMap (key-value) get set

应用

数据结构应用

缓存应用:(Redis: 官方: 10w/s)

数据 -> Mysql: 并发 写600/s 读2000/s

百万级并发

排行榜:

Redis value 列表, 有序集合 set zset

计数器 (限流、浏览器次数)

视频, 播放量, 点赞数, 关注数量。高并发, Redis

社交网络:

马士兵教育官网, 消息, 发送, 推送, 好友-》IM中台系统 (Redis<查询问题>、MQ<数据的异构问题>)

消息队列(MQ):

发布订阅, 作者开发, 参考Kafka的设计, 新增的功能。

Redis: 网上2种观点, 1、Redis可以取代传统的额关系型数据库, 2、不可以 (大厂) 数据持久化: 异步、关系型数据库。

Redis的版本号: 2.8 3.0 6.2.7

第2个版本号: 奇数,非稳定版本 (2.7 2.9 3.1) , 偶数,稳定版本

www.redis.net.cn redis中文网

启动命令

默认启动

./redis-server

带端口启动

./redis-server --port 6370

带配置文件启动

./redis-server

redis停止

在redis-cli里面使用shutdown命令,不建议使用kill -9 防止数据丢失

不建议在生产环境用keys *

查询redis键值对的数量

dbsize

查询键对是否存在

EXISTS "key"

查询键对应的值的类型

type "key"

重命名key

rename key1 key2

重命名key防止覆盖

renamenx key1 key2

随机返回key

randomkey

key:业务名称: id: 属性

数据库结构

string hash list set zset

setnx 不存在才设置

set key value nx 不存在才设置, 要求key不存在

set key value xx 存在才设置, 要求key存在

mset 批量设置key value

mset key1 value1 key2 value2 key3 value3

list操作

lpush/rpush 加入

lpop/rpop 弹出

lrange 获取数据

lrem 删除数据

ltrim 截取

lset 单独设置某个元素

lindex 根据索引获取

阻塞命令

blpop key time 没有数据,会阻塞time秒

实现消息中间件功能,用作MQ:先进先出

set操作

sadd 添加数据

smembers 查询成员

srem 删除

sismember 查询某个元素是否存在

randmember 随机返回一个元素

spop 弹出元素

sinter 交集

sinterstore 求交集并保存结果到指定key里面

sunion 并集

sdiff 差集

zset

zadd key score member 添加数据

zcard key 统计元素个数

zscore 查询分数

zrank 查询排名,从0开始

zrevrank 查询按反转排名

zrem 删除元素

zincrby 增加分数

bitmap

本质是一个二进制数组,数组每个元素都是一个bit,只存0和1

网站 1亿用户,独立访问5千万

记录活跃用户

集合类型: id --long类型 64位 = $8 * 50\,000\,000 = 400M$ (每天)

Bitmaps: 1位 = $(1/8) * 100\,000\,000 = 12.5M$

布隆过滤

通过hash计算判断是否在集合上

通过hash计算判断在数组上的不一定在集合上

本质hash

通过hash计算判断不在数组上的一定不在集合上

优化方案,

增大数组(预估适合值)

增加hash算法

缓存穿透

查不存在的key

布隆过滤解决缓存穿透问题

时间复杂度

--高并发--排名 都是定时任务去算,然后缓存结果

多少时间更新一下

Guava 依赖google的juc高级扩展包

HyperLogLog

统计大型网页每天的UV(union view)

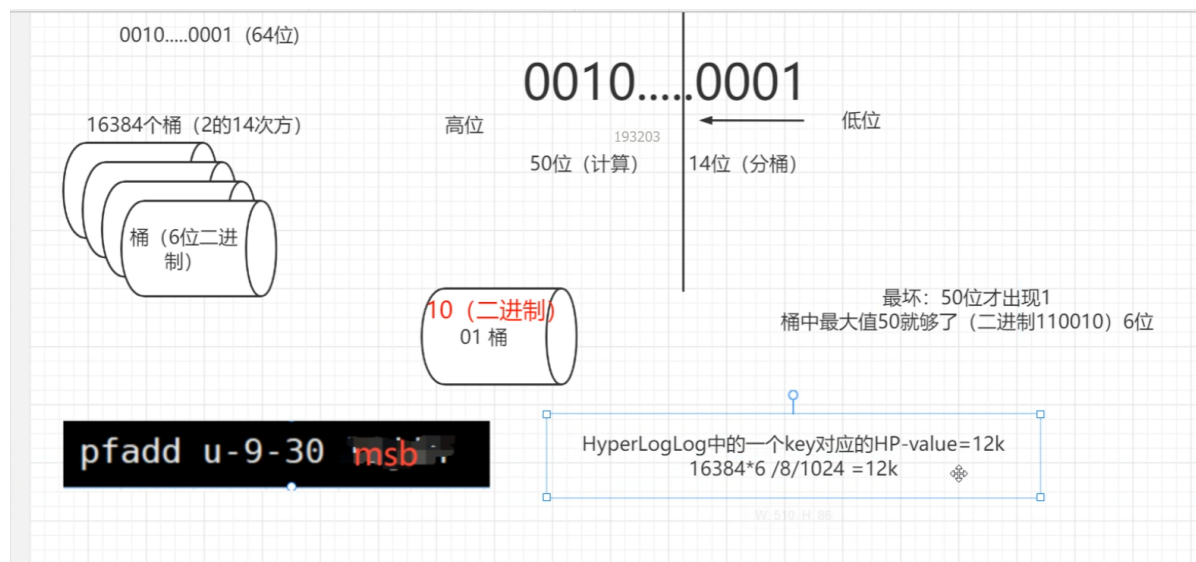
基本原理

HyperLogLog基于概率论中**伯努利试验**并结合了极大似然估算方法,并做了**分桶优化**。

pfadd 添加数据

pfcount 查询数据数量

pfmerge 合并,求并集



GEO

geoadd key member 经度 纬度

实际使用zset存储

慢查询

配置:

slowlog-log-slower-than 10000 开启10毫秒 单位是微秒(默认是0)

slowlog-max-len 128 记录方式是链表,默认长度128

指令:

slowlog get 3 查询3条记录

slowlog reset 清空记录

生产环境 slowlog-max-len 设置1000,长命令会被截断,不会非常占用内存

pipeline批量操作

一次网络请求

内核缓冲区 4k 8k

单个tcp的报文的最大值1460. 1500字节-IP头20字节,tcp头20字节=1460

事务

指令

`multi` 开启事务

命令1 命令2.....

`exec` 提交执行

只能处理语法错误,不建议使用,运行时错误,无法处理回滚

watch 监控某个key,

开启multi

最后exec时,如果key被修改,则不提交

pipeline和事务的区别

pipeline是客户端的行为,批量封装,不保证原子性

事务把指令放到缓冲区

使用lua保证原子性

指令

`eval`

lua使用redis指令

`redis.call('指令',key,val,key,val.....)`

redis可以缓存lua脚本

`script load` 脚本 //缓存脚本,并返回sha1的摘要

`evalsha` 摘要 //执行摘要对应的lua脚本

限流

固定窗口

滑动窗口

漏桶

令牌桶

发布订阅

`publish topic msg`

发布即忘

`subscribe topic` 阻塞读取

`pubsub channels` 查询频道

`pubsub numsub topic` 查询频道订阅者数量

stream (消息中间件)5.0以后

`xadd keyName * key val key2 val2 ..... 发布消息到stream的keyName主题,返回id`

*号表示使用默认id

自定义id: 数字-数字

`xrange keyName start end` 消息查询

`xlen keyName` 查询消息长度

`xdel keyName id ...` 删除指定id

消息中间件的作用,消息队列,MQ

分布式系统解耦,跨系统调用同步问题

三大功能:1解耦,2异步,3削峰填谷(流量削峰)

一般都是使用群组,一个群组,一个消息队列

群组之间都会消费相同的消息,群组内部1个消息只被消费1次

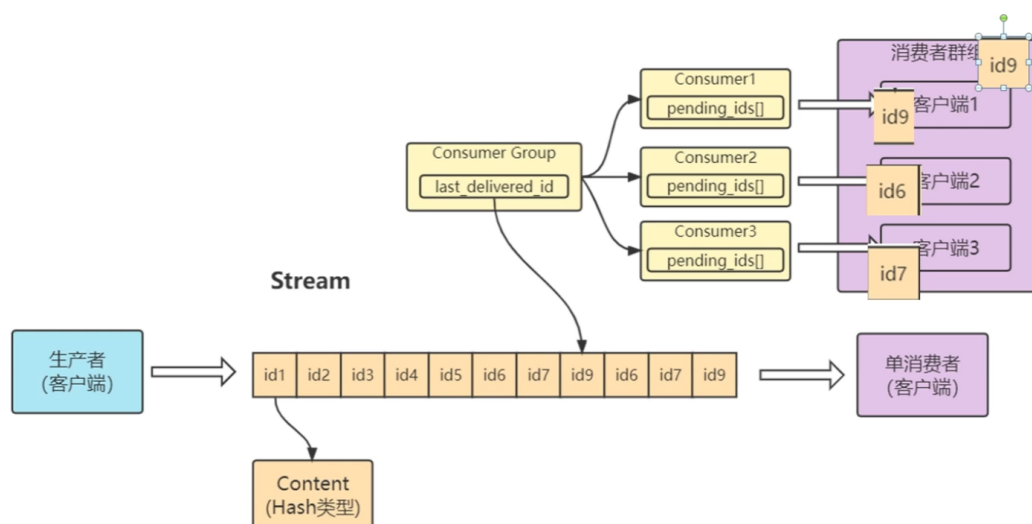
`xread count 1 streams keyName 0-0` 开始位置(id) 不包含开始位置

`xread count 1 streams keyName $` 最后的位置读取

`xread block 0 count 1 streams keyName $` 最后的位置阻塞读取 阻塞时间毫秒数0为一直阻塞



Redis Stream



创建群组

`xgroup create keyName groupName 0-0` 创建名称为groupName的群组,主题为keyName

0-0 开始位置

\$最后开始消费

`xinfo stream keyName` 查询流信息

`xinfo groups keyName` 查询群组信息

群组消费

`xreadgroup group groupName consumerName count 1 streams streamName >`

block 0 阻塞

消费确认

`xack streamName groupName id`

xclaim 故障转移

生产环境使用:

redis的stream是轻量级的,可以简单使用

如果以后需要扩展,还是使用专业的消息中间件

消息长度默认1024

redis扩容

使用渐进式rehash

双hash表,把rehash操作分散到每个请求里面处理

持久化

RDB(redis database)

save 阻塞主线程

bgsave 子线程保存

配置

save 900 1

save 300 10

save 60 10000

900秒内至少1次操作

300秒内10次操作

60秒内10000次操作

满足任意一个,触发bgsave

shutdown时,也会触发save

从节点执行全量复制操作,主节点自动执行bgsave生成一个rdb文件发给从节点

save "" 关闭rdb

dbfilename dump.rdb rdb文件名字

LZF压缩算法

AOF (append only file)

appendonly yes 开启aof

resp协议 文本协议

aof缓冲区

appendfsync always /everysec/no 不使用缓存区/每秒(默认配置)/按操作系统默认

rewrite 重写aof缩小aof体积,分析命令

aof重写缓冲区

bgrewriteaof 手动重写

AOF与RDB的加载

优先加载AOF

AOF与RDB混合持久化

触发AOF前,前段文件保存为rdb,后续的操作保存为aof

分布式锁

setnx 配合uuid上锁

lua脚本解锁,保证原子性

看门狗线程,对锁的key续约,使用lua脚本保证原子性

redis主从集群

复制和拓扑结构

从节点配置

`slaveof ip port` 设置主从结构,会清空本机数据,全量同步主节点的数据

从节点不支持写入 `slave-read-only yes`

`slaveof no one` 取消主从

`info replication` 节点信息

哨兵机制

linux: `redis-sentinel` 配置文件

win: `redis-server.exe` 配置文件 `--sentinel`

`sentinel monitor mymaster 127.0.0.1 6001 2`

2是quorum选举判断的数量

哨兵只需要监控主节点,主节点里面有从节点的信息

为了哨兵机制的高可用,哨兵一般也要搭建多个

哨兵的3个定时任务

1. 每10秒发一次info **获取节点的信息**
2. 每2秒发一次publish/subscribe **哨兵之间的发现**
3. 每隔1秒发一次ping **判断节点状态**

哨兵使用raft算法选举

redis集群

哈希分区

节点取余分区

一致性哈希分区

虚拟一致性哈希分区

为什么槽的数量是16383

1集群节点一般不会超过1000

2集群槽数据使用bitmap传输

集群的缺点

1批量操作支持有限

2事务支持有限

3根据key进行分区,无法将单个key的内容进行分区,pipeline操作支持有限

4不支持多数据库,只有0,不支持select

5复制结构只支持单层结构,不支持树形结构

缓存数据同步问题

更新缓存导致脏数据问题

修改/删除的一致性问题

不更新缓存,删除缓存,更新缓存容易出问题

延迟双删

异步延迟双删(MQ)

缓存穿透

大量查询不存在的key,使用布隆过滤

缓存击穿

热点key在大量访问时过期了,不设置过期时间,或者在过期时上锁,等查询完数据库,设置好缓存再解锁

缓存雪崩

redis不可用,需要redis高可用,服务降级,资源隔离,缓存失效时间错开

数据倾斜

热点key

大量热点key存在同一个redis

客户端监听key调用

redis发现:monitor命令

抓包TCP的包(redis服务器)

- 1、成本高: ELK packetbeat插件(redis,mysql)
- 2、机器网络波动,丢包的可能
- 3、维护成本高

二级缓存Guava-cache、hcache、jvm内存

key分散

BigKey造成网络拥堵

scan

debug object keyName

脑裂-针对哨兵模式

哨兵与主服务器网络波动导致,进行重新选举,但是客户端与主服务器是通的,在选举的过程中出现2个主

集群模式不会脑裂,集群会分配槽,只要有槽没有指派节点,则系统不可用

多级缓存架构

redis客户端工具

QuickRedis

[QuickRedis: QuickRedis 是一款永久免费的 Redis 可视化管理工具。它支持直连、哨兵、集群模式,支持亿万数量级的 key, 还有令人兴奋的 UI。QuickRedis 支持 Windows、Mac OS X 和 Linux 下运行。\(gitee.com\)](#)

[客户端下载 2.7.1 免费高速下载|百度网盘-分享无限制\(baidu.com\)](#)